

សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

មហាវិទ្យាល័យ វិទ្យាសាស្ត្រ និងបច្ចេកវិទ្យា



មុខវិជ្ជា៖

Programming Methodology in C++

មេរៀនទី៧៖

ជំពូក ៧ ការប្រើប្រាស់ Pointer to Class

ឆ្នាំទី ១ ឆមាស ទី ២

គណៈកម្មការអភិវឌ្ឍន៍កម្មវិធីសិក្សា

© 2024

មតិកា

- ❑ 1. ការបង្កើត និង ប្រើប្រាស់ Classes និង Objects
- ❑ 2. ការប្រើប្រាស់ Constructor និង Destructor
- ❑ 3. ការប្រើប្រាស់ Pointer to Class
- ❑ 4. ការប្រើប្រាស់ Class Defined with Structure , Union
- ❑ 5. ការប្រើប្រាស់ Overloading the Operators, Static Member
- ❑ 6. ការប្រើប្រាស់ FriendShip និង Inheritance
- ❑ 7. ការប្រើប្រាស់ Single និង Multiple Inheritance
- ❑ 8. ការប្រើប្រាស់ Pointer to Base Class Abstract Base Class

មតិកា

- ❑9. ការប្រើប្រាស់ Function Template និង Class Template
- ❑10. ការប្រើប្រាស់ Virtual Member , Template Specialization
- ❑11. ការប្រើប្រាស់ NameSpace និង ដោះស្រាយ Exception
- ❑ 12. ការប្រើ ការបំបែកប្រភេទទិន្នន័យ
- ❑១៣. ការកំណត់ Preprocessor Directives ការប្រើ Input/ output with Files.
- ❑14. ការប្រតិបត្តិរបៀប Read និង Write Files

4. ការប្រើប្រាស់ Class Defined with Structure , Union

- 1.1. អ្វីទៅ ហៅថា Structure?
- 1.2. ការប្រើប្រាស់ Structure.
- 1.3. អ្វីទៅ ហៅ ថា Union?
- 1.4. ការប្រើប្រាស់ Union.
- Example : Coding .

១.1. អ្វីទៅ ហៅថា Structure?

- **Structure** គឺសំដៅលើការប្រមូលផ្តុំនូវទិន្នន័យ ឬ ព័ត៌មានផ្សេងៗ ដាក់ជាបណ្តុំមួយ ដោយប្រើប្រាស់នូវ Keyword Struct ដើម្បីបង្កើត វាឡើង។ Structure វាអាចប្រើប្រាស់នូវ Keyword struct ដើម្បីបង្កើតវាឡើង។ Structure វាអាចប្រើប្រាស់នូវទិន្នន័យទាំងនោះបានត្រូវប្រើប្រាស់តាមរយៈការបង្កើត Object របស់ Structure ដែលវាអាចបង្កើតចេញពី ឈ្មោះ structure តែម្តង។
- **Structures** គឺមានប្រយោជន៍ក្នុងករណីដែលអ្នកចង់អោយទិន្នន័យច្រើនប្រភេទទាំងអស់ក្នុងធាតុមួយ តែប៉ុណ្ណោះ ដែលធ្វើឱ្យក្នុងមានលក្ខណៈត្រឹមត្រូវ និងងាយស្រួលសម្រាប់ការគ្រប់គ្រងទិន្នន័យ។

Structure (struct) មានលក្ខណៈពិសេស

- ប្រភេទទិន្នន័យផ្ទាល់ខ្លួន (User-defined Data Type)
- ការចូលប្រើតាមឧបករណ៍ . (Access via Dot Operator)
- អាចផ្គុំទិន្នន័យច្រើនប្រភេទ (Multiple Data Types)
- អាចប្រើជាមួយអរ្យាយ (Array of Structures)
- អាចបញ្ជូនទៅកាន់ Functions (Passing to Functions)
- ការគ្រប់គ្រងទិន្នន័យបានល្អ (Data Encapsulation)
- Public by Default
- មូលដ្ឋានសម្រាប់ការបង្កើត Classes
- អាចប្រើតួអក្សរ (Structure Pointers)

១.២. ការប្រើប្រាស់ Structure

❖ នម្រង់ទូទៅរបស់ Structure (Syntax)

```
struct StructureName {  
    DataType1 member1;  
    DataType2 member2;  
    // អាចបន្ថែមធាតុទិន្នន័យផ្សេងទៀត  
};
```

១.២. ការប្រើប្រាស់ Structure

- ការបង្កើត អថេរបស់ Structure:

```
StructureName variableName;
```

- ការបញ្ចូលតម្លៃទៅធាតុសមាជិក៖

អ្នកអាចបញ្ចូលតម្លៃទៅក្នុងធាតុសមាជិកនៃ Structure ដោយប្រើ Operator ‘.’ (dot).

```
variableName.member1 = value1;  
variableName.member2 = value2;
```


Coding Structure

```

1  #include <iostream>
2  using namespace std;
3
4  // កំណត់ Structure ឈ្មោះ Student
5  struct Student {
6      int id;
7      string name;
8      float score;
9  };
10
11 int main() {
12     // បង្កើតអថេររបស់ Structure Student
13     Student student1;
14
15     // បញ្ចូលតម្លៃទៅធាតុសមាជិក
16     student1.id = 1;
17     student1.name = "Alice";
18     student1.score = 89.5;
19
20     // បង្ហាញតម្លៃរបស់ student1
21     cout << "ID: " << student1.id << endl;
22     cout << "Name: " << student1.name << endl;
23     cout << "Score: " << student1.score << endl;
24
25     return 0;
26 }

```

STDIN

Input for the program (Optional)

Output:

ID: 1

Name: Alice

Score: 89.5

Coding Structure ជាមួយ Array

```
#include <iostream>
using namespace std;

// កំណត់ Structure Student
struct Student {
    int id;
    string name;
    float score;
};

int main() {
    // បង្កើតអារេនៃ Structure Student ដែលមាន 3 អថេរ
    Student students[3];

    // បញ្ចូលតម្លៃទៅអថេរនីមួយៗ
    students[0] = {1, "Alice", 89.5};
    students[1] = {2, "Bob", 92.0};
    students[2] = {3, "Charlie", 85.7};

    // បង្ហាញតម្លៃ
    for(int i = 0; i < 3; i++) {
        cout << "ID: " << students[i].id << ", ";
        cout << "Name: " << students[i].name << ", ";
        cout << "Score: " << students[i].score << endl;
    }

    return 0;
}
```

STDIN

Input for the program (Optional)

Output:

ID: 1, Name: Alice, Score: 89.5

ID: 2, Name: Bob, Score: 92

ID: 3, Name: Charlie, Score: 85.7

Coding Structure ជាមួយ Function

```
#include <iostream>
using namespace std;

// កំណត់ Structure Student
struct Student {
    int id;
    string name;
    float score;
};

// Function សម្រាប់បង្ហាញព័ត៌មានសិស្ស
void displayStudent(Student s) {
    cout << "ID: " << s.id << endl;
    cout << "Name: " << s.name << endl;
    cout << "Score: " << s.score << endl;
}

int main() {
    // បង្កើតអថេររបស់ Structure Student
    Student student1 = {1, "Alice", 89.5};

    // បញ្ជូន Structure ទៅកាន់ function
    displayStudent(student1);

    return 0;
}
```

STDIN

Input for the program (Optional)

Output:

ID: 1
Name: Alice
Score: 89.5

១.3. អ្វីទៅ ហៅ ថា Union?

- **Union** ក្នុង C++ គឺជាប្រភេទទិន្នន័យផ្ទាល់ខ្លួនដែលស្រដៀងទៅនឹង Structure (struct) ប៉ុន្តែមានលក្ខណៈពិសេសដែលធ្វើឱ្យវាខុសគ្នា។ Union អនុញ្ញាតឱ្យតម្លៃទិន្នន័យនានានៅក្នុងសមាជិកទាំងអស់ត្រូវបានរក្សាទុកនៅក្នុងទីតាំង (memory location) តែមួយគត់។

1.4. ការប្រើប្រាស់ Union.

❖ ទម្រង់ទូទៅរបស់ Structure (Syntax)

```
union UnionName {  
    DataType1 member1;  
    DataType2 member2;  
    // អាចបន្ថែមធាតុទិន្នន័យផ្សេងទៀត  
};
```

លក្ខណៈពិសេសនៃ Union:

- **ការចំណាយ Memory តិច (Memory-efficient):**

បើប្រៀបធៀបនឹង Structure, Union ចំណាយអនុស្សាវរីយ៍តិចជាងដោយសារតែសមាជិកទាំងអស់ចែករំលែក memory location តែមួយ។ Memory location នោះមានទំហំស្មើនឹងធាតុដែលមានទំហំធំបំផុតនៅក្នុង Union ។

- **ការចូលប្រើតាម . (dot operator):**

ដូច Structure អ្នកអាចចូលប្រើសមាជិកនៃ Union ដោយប្រើ . (dot operator) ។

- **តម្លៃតែសមាជិកតែមួយ:** នៅពេលណាមួយ អ្នកអាចកំណត់តម្លៃសម្រាប់តែសមាជិក Union មួយប៉ុណ្ណោះ។ ការចូលប្រើសមាជិកផ្សេងទៀតនឹងធ្វើឱ្យតម្លៃមុនបាត់បង់។

- **ប្រើសម្រាប់តម្លៃផ្សេងៗដែលមិនត្រូវការ:** Union មានប្រយោជន៍នៅពេលដែលអ្នកចង់ផ្ទុកតម្លៃប្រភេទផ្សេងៗដែលមិនត្រូវការប្រើប្រាស់ជាមួយគ្នាក្នុងពេលតែមួយ។

1.4. ការប្រើប្រាស់ Union.

ការប្រើប្រាស់សម្រាប់ Union:

- **Union** ប្រើប្រាស់នៅពេលដែលអ្នកចង់សន្សំ memory ដោយចែករំលែកទីតាំង memory ក្នុងករណីដែលសមាជិកមិនត្រូវបានប្រើប្រាស់ក្នុងពេលដំណើរការដែលដូចគ្នា។
- វាមានប្រយោជន៍នៅក្នុងប្រព័ន្ធដែលមាន memory តិចឬមានកំរិត memory ខ្ពស់សម្រាប់ការគ្រប់គ្រងតម្លៃដែលមានប្រភេទផ្សេងៗគ្នា។

Coding Union

```
#include <iostream>
using namespace std;

union Data {
    int intValue;
    float floatValue;
    char charValue;
};

int main() {
    Data data;

    // បញ្ចូលតម្លៃទៅ intValue
    data.intValue = 10;
    cout << "Integer: " << data.intValue << endl;

    // បញ្ចូលតម្លៃទៅ floatValue
    data.floatValue = 20.5;
    cout << "Float: " << data.floatValue << endl;

    // បញ្ចូលតម្លៃទៅ charValue
    data.charValue = 'A';
    cout << "Character: " << data.charValue << endl;

    // សូមចំណាំ៖ ការបញ្ចូលតម្លៃទៅក្នុងសមាជិកមួយ នឹងធ្វើឱ្យតម្លៃនៃសមាជិកមុនបាត់បង់
    cout << "Integer: " << data.intValue << endl; // តម្លៃនេះ

    return 0;
}
```

STDIN

Input for the program (Optional)

Output:

```
Integer: 10
Float: 20.5
Character: A
Integer: 1101267009
```




សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

ASIA EURO UNIVERSITY

សិក្សាស្រាវជ្រាវ គ្រប់ទីកន្លែង

អរគុណ

សម្រាប់ការយកចិត្តទុកដាក់

<https://elearning.aeu.cloud>